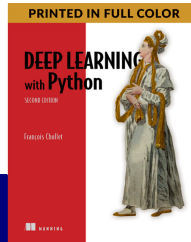


CS 59300

Application of Deep Learning Introduction to Deep Learning and Neural Networks



Instructor: Dr. Zesheng Chen

PURDUE UNIVERSITY FORT WAYNE Outline

- Deep Learning: Why and How
- “Hello World” Example
- Considerations for Neural Networks
- “Classify Movie Reviews as Positive or Negative” Example

2

PURDUE
UNIVERSITY
FORT WAYNE

Why Deep Learning?

- In ML, what makes DL different?
 - Achieves better **performance** for many problems
 - Completely **automates** what used to be the most crucial step in ML
 - **Feature Engineering**
 - Allows a model to learn all layers of representation **jointly** at the same time, rather than in succession (greedily)

3

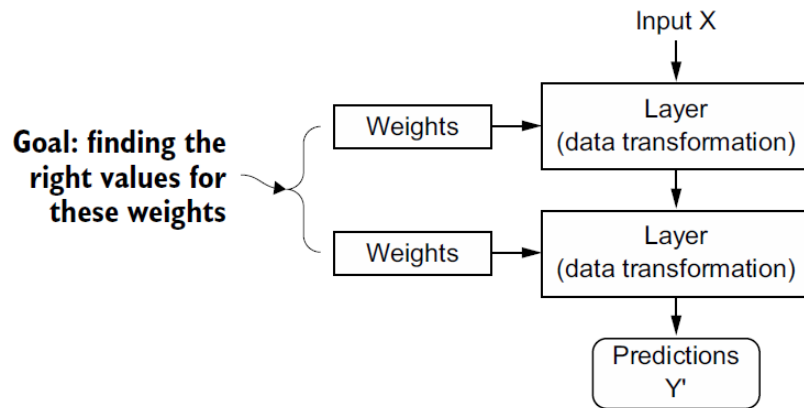
PURDUE
UNIVERSITY
FORT WAYNE

How Deep Learning Works?

- Remind: From ML (three main components)
 - Weights or parameters
 - Loss function or objective function or cost function
 - Optimizer
 - Gradient decent

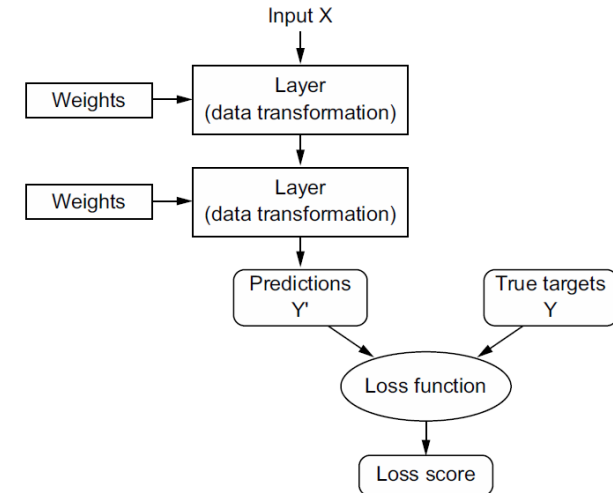
4

How Deep Learning Works?



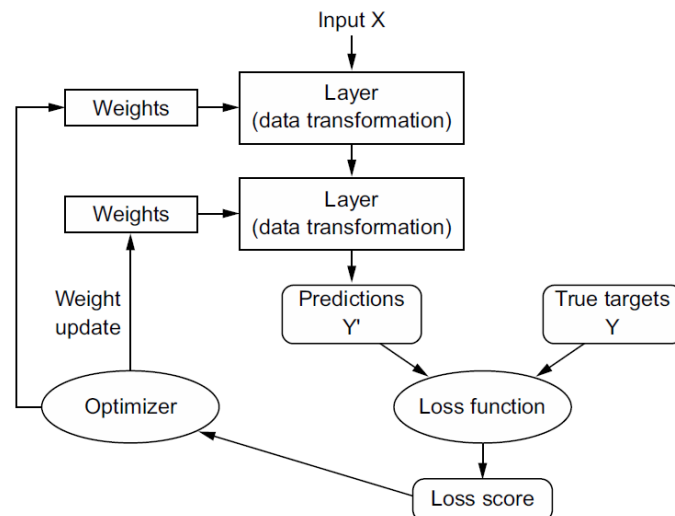
5

How Deep Learning Works?



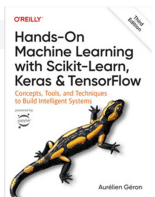
6

How Deep Learning Works?



7

Perceptron

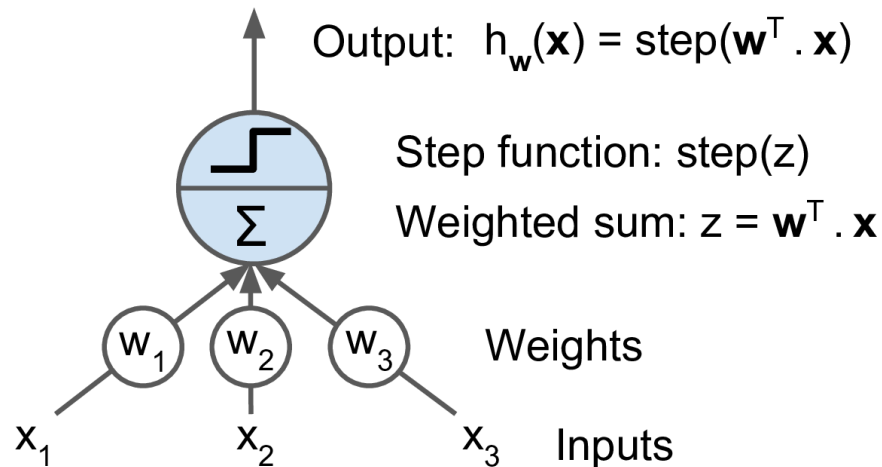


- Invented in 1957 by Frank Rosenblatt
- Is one of the **simplest** artificial neural networks (ANN) architecture
- Is based on a **linear threshold unit (LTU)**
 - The inputs and output are numbers (instead of binary on/off values) and each input connection is associated with a weight.

8



Linear Threshold Unit (LTU)



9



Common Step Functions Used in Perceptrons

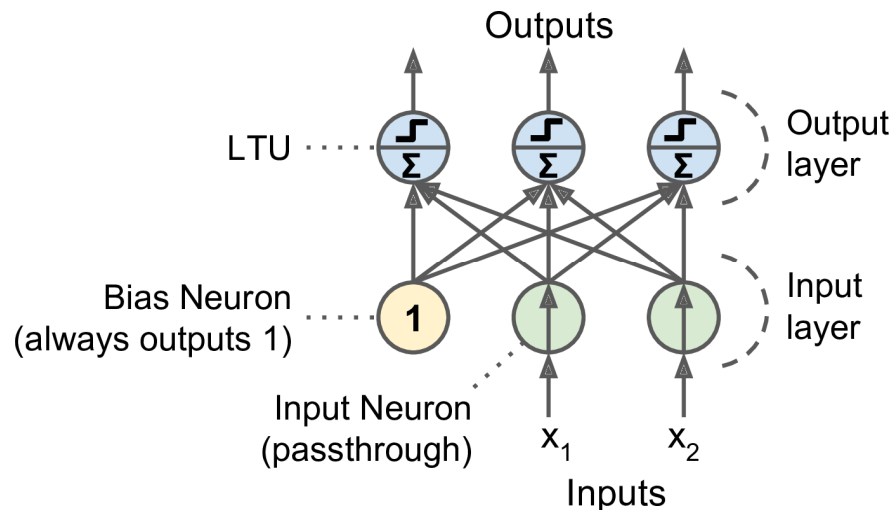
$$\text{heaviside}(z) = \begin{cases} 0 & \text{if } z < 0 \\ 1 & \text{if } z \geq 0 \end{cases}$$

$$\text{sgn}(z) = \begin{cases} -1 & \text{if } z < 0 \\ 0 & \text{if } z = 0 \\ +1 & \text{if } z > 0 \end{cases}$$

10



Perceptron



11



Computation Equation

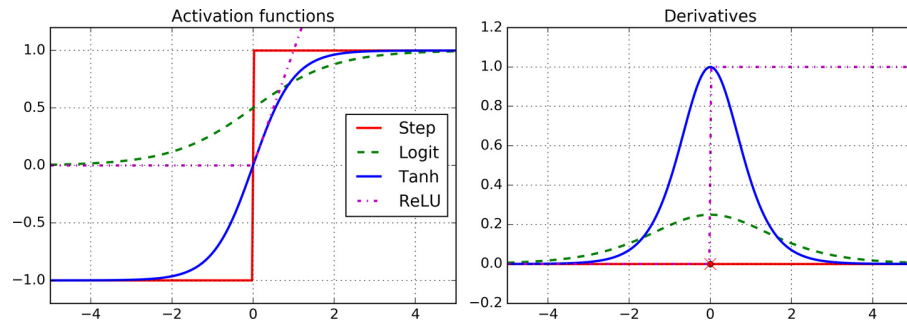
■ $h_{\mathbf{W}, \mathbf{b}}(\mathbf{X}) = \phi(\mathbf{X}\mathbf{W} + \mathbf{b})$ `output = relu(dot(input, W) + b)`

- $\mathbf{X}$: matrix of input features. One row per instance and one column per feature.
- $\mathbf{W}$ (weight matrix): contains all the connection weights except for the ones from the bias neuron. One row per input neuron and one column per artificial neuron in the layer.
- $\mathbf{b}$ (bias vector): contains all the connection weights between the bias neuron and the artificial neurons. One bias term per artificial neuron.
- ϕ : activation function.

12



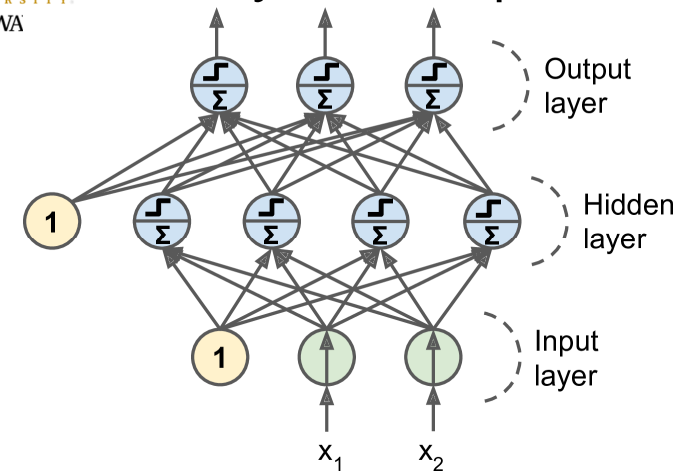
Activation Functions and Their Derivatives



13



Multilayer Perceptron



- DNN: when an ANN contains a deep stack of hidden layers, it is called a **deep neural network**.

14



More on Perceptron and Multilayer Perceptron

- MIT Introduction to Deep Learning | 6.S191
- <http://introtodeeplearning.com/>
- http://introtodeeplearning.com/slides/6S191_MIT_DeepLearning_L1.pdf
- https://www.youtube.com/watch?v=alfdl7S6wCY&list=PLtBw6njQRU-rwp5_7C0oIVt26ZqjG9NI&index=1

15



Outline

- Deep Learning: Why and How
- **“Hello World” Example**
- Considerations for Neural Networks
- “Classify Movie Reviews as Positive or Negative” Example

16



“Hello World” Example

- MNIST dataset
 - Grayscale images of handwritten digits (28 x 28 pixels)
 - 60,000 training images and 10,000 test images
- Goal: classify images into their 10 categories (0 through 9)



17



Terminology

- Class
 - A category in a classification problem
- Samples
 - Data points
- Label
 - The class associated with a specific sample

18



Steps

- Obtain the data
- Build the DL model
- Prepare the data
- Train / fit the model
- Evaluate the model on new data or test data

19



Demo

20

Outline

- Deep Learning: Why and How
- “Hello World” Example
- **Considerations for Neural Networks**
- “Classify Movie Reviews as Positive or Negative” Example

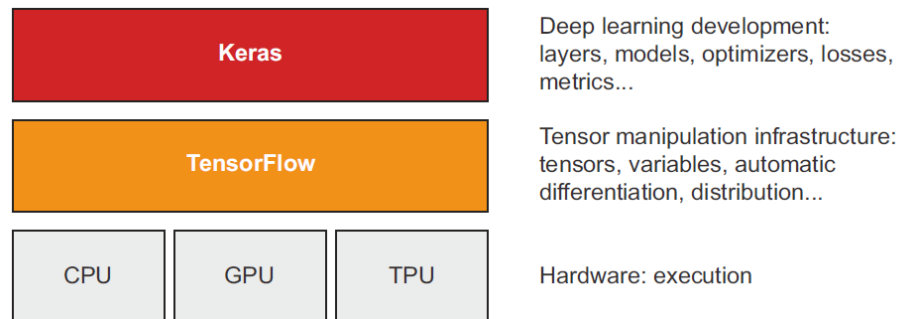
21

Main Libraries for DL

- Keras with TensorFlow
- PyTorch

22

Relationship between TF and Keras



23

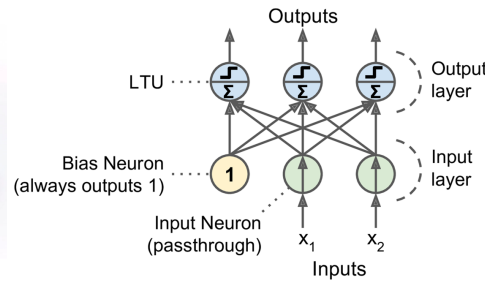
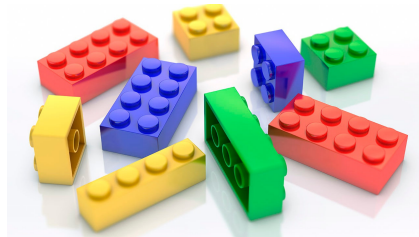
Layers

- Layers are the building blocks of deep learning
- Layer's weights contain the NN's knowledge
- Different types for different input data
 - Vector data
 - Fully connected or dense layers
 - Sequence data
 - Recurrent layers such as LSTM layer or 1D convolution layers (Conv1D)
 - Image data
 - 2D convolution layers (Conv2D)

24



Layers are the LEGO Bricks of Deep Learning



25



Configuring the Learning Process

- *Loss function (objective function)*
 - The quantity that will be **minimized** during training. It represents a measure of **success** for the task.
- *Optimizer*
 - Determines how the network will be **updated** based on the loss function and implements a **specific** variant of stochastic gradient descent (SGD).
- *Metrics*
 - The measures of **success** you want to monitor during training and validation, such as classification accuracy. **Unlike** the loss, training will not optimize directly for these metrics. As such, metrics don't need to be **differentiable**.

26



The **compile()** Method

```
model = keras.Sequential([keras.layers.Dense(1)])
model.compile(optimizer="rmsprop",
              loss="mean_squared_error",
              metrics=["accuracy"])
```

```
model.compile(optimizer=keras.optimizers.RMSprop(),
              loss=keras.losses.MeanSquaredError(),
              metrics=[keras.metrics.BinaryAccuracy()])
```

```
model.compile(optimizer=keras.optimizers.RMSprop(learning_rate=1e-4),
              loss=my_custom_loss,
              metrics=[my_custom_metric_1, my_custom_metric_2])
```

27



Losses

- CategoricalCrossentropy
- SparseCategoricalCrossentropy
- BinaryCrossentropy
- MeanSquaredError
- KLDivergence
- CosineSimilarity
- Etc.

28

Picking a Loss Function

- Choosing the **right** loss function for the **right** problem is extremely important.
- If the objective doesn't fully **correlate** with success for the task at hand, your network will end up doing things you may not have wanted.
 - AI: "maximizing the average well-being of all humans alive."
- There are **simple** guidelines you can follow to choose the correct loss.
 - binary crossentropy for a two-class classification problem
 - categorical crossentropy for a many-class classification problem

29

Optimizers

- SGD (with or without momentum)
- RMSprop
- Adam
- Adagrad
- Etc.

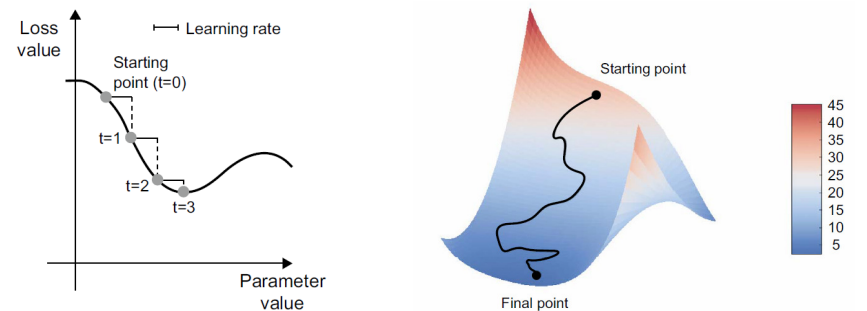
30

Gradient Descent

- Batch gradient descent
 - Batch size == the size of all available data
- Stochastic gradient descent (SGD)
 - Batch size < the size of all available data
 - Stochastic: each batch of data is drawn at **random**
 - **True SGD**: Single sample
 - **Mini-batch SGD**: size > 1

31

Low-Dimension vs. High-Dimension

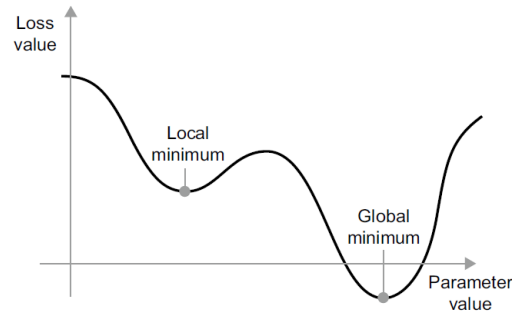


- **Note: it's good to keep in mind that the intuitions you develop through these low-dimensional representations may not always be accurate in practice. This has historically been a source of issues in the world of deep learning research.**

32



Local Minimum vs. Global Minimum



- To avoid the local minimum, consider to use SGD with momentum, as well as Adagrad, RMSprop, and several others.

33



Metrics

- CategoricalAccuracy
- SparseCategoricalAccuracy
- BinaryAccuracy
- AUC
- Precision
- Recall
- F1 Score
- Etc.

34



Two Main Issues for DL

- Underfitting
- Overfitting

35



Monitoring Loss and Metrics on Validation Data

- The goal is to obtain models that perform well **in general**, and particularly on data points that the model has **never** encountered before.
- **Memorizing** a mapping between your training samples and their targets is **useless** for the task of predicting targets for data the model has never seen before.
- It's standard practice to reserve a subset of the training data as **validation data**.
 - You won't be training the model on this data, but you will use it to compute a loss value and metrics value.

36



Using the validation_data Argument in fit() Method

```
indices_permutation = np.random.permutation(len(inputs))
shuffled_inputs = inputs[indices_permutation]
shuffled_targets = targets[indices_permutation]

num_validation_samples = int(0.3 * len(inputs))
val_inputs = shuffled_inputs[:num_validation_samples]
val_targets = shuffled_targets[:num_validation_samples]
training_inputs = shuffled_inputs[num_validation_samples:]
training_targets = shuffled_targets[num_validation_samples:]
model.fit(
    training_inputs,
    training_targets,
    epochs=5,
    batch_size=16,
    validation_data=(val_inputs, val_targets)
)
```

37



Outline

- Deep Learning: Why and How
- “Hello World” Example
- Considerations for Neural Networks
- “Classify Movie Reviews as Positive or Negative” Example

38



Classifying movie reviews

- Goal: classify movie reviews as **positive or negative**, based on the text content of the reviews.
- **IMDB** dataset
 - A set of **50,000** highly polarized reviews from the Internet Movie Database (IMDB).
 - They’re split into 25,000 reviews for **training** and 25,000 reviews for **testing**.
 - Each set consists of **50% negative and 50% positive** reviews.

39



Questions to Be Considered

- Supervised learning or unsupervised learning?
 - Supervised learning with training data
- Classification or regression?
 - Classification
- Binary classification, multiclass classification, multilabel classification, or multioutput classification?
 - Binary classification

40

Questions to Be Considered

■ Data preprocessing needed?

```
from tensorflow.keras.datasets import imdb
(train_data, train_labels), (test_data, test_labels) = imdb.load_data(
    num_words=10000)
```

```
>>> train_data[0]
[1, 14, 22, 16, ... 178, 32]
>>> train_labels[0]
1
```

```
print(len(train_data[0])), print(len(train_data[1]))
```

```
218
189
```

41

Two Possible Solutions

■ Padding

- ☐ Pad the lists so that they all have the **same** length.
- ☐ Each sample has the vector length of “max_length”.

■ Multi-hot encoding

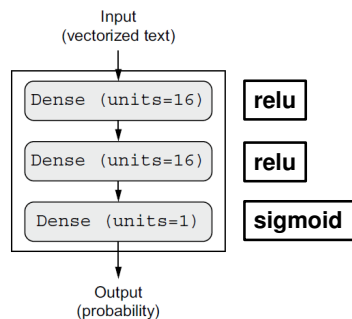
- ☐ Turn lists into vectors of 0s and 1s.
- ☐ For instance, turn the sequence [8, 5] into a 10,000-dimensional vector that would be all 0s except for indices 8 and 5, which would be 1s.
- ☐ Use this solution in the example.

42

Questions to Be Considered

■ DL NN model structure?

- ☐ How many layers to use?
- ☐ How many units to choose for each layer?
- ☐ What is an activation function for a layer?



43

Relu and Sigmoid Functions

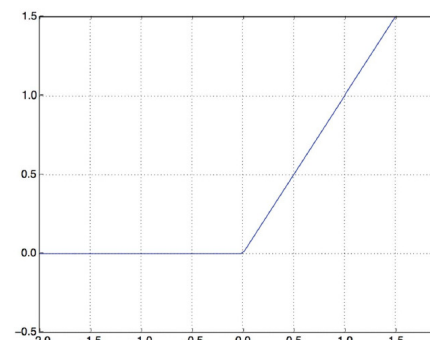


Figure 4.2 The rectified linear unit function

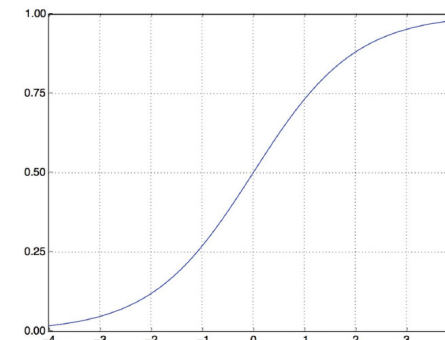


Figure 4.3 The sigmoid function

44



Questions to Be Considered

- What is the loss function?
 - Study a **binary classification** problem and the output of the model is a **probability**.
 - Best to use the **binary_crossentropy** loss.
 - It isn't the only viable choice: for instance, you **could** use `mean_squared_error`.
 - **Crossentropy** is a quantity from the field of information theory that measures the distance between probability distributions

45



Questions to Be Considered

- What is the optimizer?
 - Go with **rmsprop**, which is usually a good **default** choice for virtually any problem.
- How many epochs?
 - Begin with 20.
 - But if epochs are too big, it can cause **overfitting**.
 - How to know the **correct** value of epochs?
- What is the batch size?
 - Use 512 in this example.

46



Questions to Be Considered

- What is the baseline?
 - Each set consists of **50% negative and 50% positive** reviews.
 - Use a **random classifier**. What is the accuracy?
 - 50%!
 - **Guess all positive**. What is the accuracy?
 - 50%!

47



Code Structure

- Load the dataset
- Prepare the data
 - Input data and output data
- Build the model
 - Define and compile the model
- Validate the model
 - Fit with a validation set
 - Refine the model
- Predict new data

48



Wrapping Up (Binary Classification)

- **Preprocess** on raw data in order to be able to feed it—as tensors—into a neural network.
- Stacks of Dense layers with relu activations can solve a **wide** range of problems.
- In a binary classification problem (two output classes), your model should end with a Dense layer with one unit and a **sigmoid** activation.
 - The output of the model should be a scalar between 0 and 1, encoding a **probability**.

49



Wrapping Up (Binary Classification)

- With such a scalar sigmoid output on a binary classification problem, the loss function you should use is **binary_crossentropy**.
- The **rmsprop** optimizer is generally a good enough choice, whatever your problem.
- Be sure to always **monitor** performance on data that is outside of the training set.
 - Deal with the **overfitting** issue

50



Summary

- Understand the building blocks of NN
 - **Perceptron** and **Multilayer Perception**
 - Three key concepts
 - **Loss**: the quantity to minimize during training.
 - **Optimizer**: specifies the exact way in which the gradient of the loss will be used to update parameters.
 - **Metrics**: the measures of success you want to monitor during training and validation

51

